

Java Important questions and answer

OOPs Concepts (1-20)

1. What is OOP?

Answer: Programming paradigm based on objects and classes.

2. What are the four pillars of OOP?

Answer: Encapsulation, Inheritance, Polymorphism, Abstraction.

3. What is a Class?

Answer: Blueprint for creating objects.

4. What is an Object?

Answer: Instance of a class.

5. What is Encapsulation?

Answer: Binding data and methods together.

6. Advantage of Encapsulation?

Answer: Data security and maintainability.

7. What is Inheritance?

Answer: Acquiring properties from parent class.

8. Types of Inheritance in Java?

Answer: Single, Multilevel, Hierarchical.

9. Why Multiple Inheritance not supported?

Answer: To avoid Diamond Problem.

10. What is Polymorphism?

Answer: One interface, multiple implementations.

11. Types of Polymorphism?

Answer: Compile-time and Runtime.

12. What is Method Overloading?

Answer: Same method, different parameters.

13. What is Method Overriding?

Answer: Child class redefining parent method.

14. What is Abstraction?

Answer: Hiding implementation details.

15. Abstract Class vs Interface?

Answer: Abstract class can have concrete methods; interface mainly defines behavior.

16. Can abstract class have constructor?

Answer: Yes.

17. Can interface have methods?

Answer: Yes (default/static methods from Java 8).

18. What is super keyword?

Answer: Refers to parent class object.

19. What is this keyword?

Answer: Refers to current object.

20. Difference between Class and Object?

Answer: Class is blueprint; object is instance.

Collections (21-45)**21. What is Collection Framework?**

Answer: Architecture for storing and manipulating data.

22. What are main interfaces?

Answer: List, Set, Queue, Map.

23. What is List?

Answer: Ordered collection allowing duplicates.

24. What is Set?

Answer: Collection that doesn't allow duplicates.

25. What is Queue?

Answer: FIFO data structure.

26. What is Map?

Answer: Key-value pair collection.

27. ArrayList vs LinkedList?

Answer: ArrayList faster for retrieval; LinkedList faster for insertion.

28. HashSet vs TreeSet?

Answer: HashSet unordered; TreeSet sorted.

29. HashMap vs Hashtable?

Answer: HashMap unsynchronized; Hashtable synchronized.

30. HashMap allows null?

Answer: Yes.

31. TreeMap allows null key?

Answer: No.

32. What is Iterator?

Answer: Used to traverse collections.

33. What is Comparable?

Answer: Natural sorting.

34. What is Comparator?

Answer: Custom sorting.

35. Difference between Comparable and Comparator?

Answer: Comparable modifies class; Comparator external sorting.

36. What is ConcurrentModificationException?

Answer: Occurs when collection modified during iteration.

37. What is PriorityQueue?

Answer: Queue based on priority.

38. What is Vector?

Answer: Synchronized dynamic array.

39. What is Stack?

Answer: LIFO data structure.

40. What is LinkedHashMap?

Answer: Maintains insertion order.

41. What is LinkedHashSet?

Answer: Ordered HashSet.

42. Internal working of HashMap?

Answer: Uses buckets and hashing.

43. What is Load Factor?

Answer: Determines resizing threshold.

44. Default Load Factor?

Answer: 0.75

45. Initial Capacity of HashMap?

Answer: 16

Exception Handling (46-60)

46. What is Exception?

Answer: Unexpected event during execution.

47. What is Exception Handling?

Answer: Managing runtime errors.

48. try block?

Answer: Contains risky code.

49. catch block?

Answer: Handles exceptions.

50. finally block?

Answer: Always executes.

51. Checked Exception?

Answer: Compile-time checked exception.

52. Unchecked Exception?

Answer: Runtime exception.

53. Examples of Checked Exception?

Answer: IOException, SQLException.

54. Examples of Unchecked Exception?

Answer: NullPointerException.

55. What is throw?

Answer: Explicitly throws exception.

56. What is throws?

Answer: Declares exception.

57. Can multiple catch blocks exist?

Answer: Yes.

58. What is custom exception?

Answer: User-defined exception.

59. What is Stack Trace?

Answer: Sequence of method calls leading to exception.

60. Difference between Error and Exception?

Answer: Errors are system-level; exceptions are application-level.

Multithreading (61-75)

61. What is Thread?

Answer: Lightweight subprocess.

62. How to create thread?

Answer: Extend Thread or implement Runnable.

63. Runnable vs Thread?

Answer: Runnable preferred.

64. What is start()?

Answer: Starts new thread.

65. What is run()?

Answer: Contains thread logic.

66. What is Synchronization?

Answer: Controls shared resource access.

67. What is Deadlock?

Answer: Threads waiting forever.

68. What is Race Condition?

Answer: Multiple threads modifying same data.

69. What is Thread Priority?

Answer: Determines scheduling preference.

70. What is sleep()?

Answer: Pauses thread temporarily.

71. What is join()?

Answer: Waits for thread completion.

72. What is daemon thread?

Answer: Background service thread.

73. What is Executor Framework?

Answer: Thread pool management.

74. What is Callable?

Answer: Returns result from thread.

75. Future Interface?

Answer: Holds asynchronous result.

JDBC (76–90)

76. What is JDBC?

Answer: Java Database Connectivity API.

77. JDBC Architecture?

Answer: Java Application → JDBC API → Driver Manager → Database.

78. Steps in JDBC?

Answer: Load Driver → Connect → Create Statement → Execute Query → Close Connection.

79. What is DriverManager?

Answer: Manages JDBC drivers and connections.

80. What is Connection?

Answer: Session between Java application and database.

81. What is Statement?

Answer: Executes SQL queries.

82. What is PreparedStatement?

Answer: Precompiled SQL statement.

83. Why PreparedStatement?

Answer: Faster and prevents SQL Injection.

84. What is CallableStatement?

Answer: Used to call stored procedures.

85. What is ResultSet?

Answer: Stores query results.

86. What is SQL Injection?

Answer: Malicious SQL code insertion attack.

87. What is Batch Processing?

Answer: Execute multiple queries together.

88. What is Transaction?

Answer: Group of SQL operations treated as one unit.

89. ACID Properties?

Answer: Atomicity, Consistency, Isolation, Durability.

90. Auto Commit?

Answer: Automatically commits every SQL statement.

Servlet & JSP (91–105)

91. What is Servlet?

Answer: Server-side Java component.

92. Servlet Life Cycle?

Answer: init(), service(), destroy().

93. What is GenericServlet?

Answer: Protocol-independent servlet.

94. What is HttpServlet?

Answer: Handles HTTP requests.

95. GET vs POST?

Answer: GET retrieves data; POST sends data.

96. What is JSP?

Answer: Java Server Pages for dynamic web content.

97. JSP Life Cycle?

Answer: Translation → Compilation → Initialization → Execution → Destruction.

98. What are JSP Directives?

Answer: page, include, taglib.

99. What is RequestDispatcher?

Answer: Forwards request to another resource.

100. What is Session Tracking?

Answer: Maintaining user state.

101. Session vs Cookie?

Answer: Session server-side; Cookie client-side.

102. What is URL Rewriting?

Answer: Session tracking using URL parameters.

103. What is Filter?

Answer: Intercepts requests and responses.

104. What is Listener?

Answer: Monitors application events.

105. MVC Architecture?

Answer: Model + View + Controller.

Hibernate (106–120)

106. What is Hibernate?

Answer: ORM framework.

107. ORM Full Form?

Answer: Object Relational Mapping.

108. Advantages of Hibernate?

Answer: Less SQL, portability, productivity.

109. What is Entity Class?

Answer: Java class mapped to database table.

110. What is SessionFactory?

Answer: Creates Session objects.

111. What is Session?

Answer: Performs CRUD operations.

112. What is Transaction in Hibernate?

Answer: Unit of work.

113. What is HQL?

Answer: Hibernate Query Language.

114. What is Criteria API?

Answer: Programmatic query creation.

115. What is Lazy Loading?

Answer: Load data only when needed.

116. What is Eager Loading?

Answer: Load data immediately.

117. First-Level Cache?

Answer: Session-level cache.

118. Second-Level Cache?

Answer: SessionFactory-level cache.

119. get() vs load()?

Answer: get() immediate fetch, load() lazy fetch.

120. Hibernate Annotations?

Answer: @Entity, @Id, @Column, @Table.

Spring Core (121–145)

121. What is Spring Framework?

Answer: Enterprise application framework.

122. What is IoC?

Answer: Inversion of Control.

123. What is Dependency Injection?

Answer: Injecting object dependencies.

124. Types of DI?

Answer: Constructor and Setter Injection.

125. What is Spring Bean?

Answer: Object managed by Spring Container.

126. Bean Scope Types?

Answer: Singleton, Prototype, Request, Session.

127. What is ApplicationContext?

Answer: Advanced Spring Container.

128. BeanFactory vs ApplicationContext?

Answer: ApplicationContext provides extra features.

129. What is @Component?

Answer: Generic Spring Bean.

130. What is @Service?

Answer: Service Layer Bean.

131. What is @Repository?

Answer: DAO Layer Bean.

132. What is @Controller?

Answer: Handles web requests.

133. What is @Autowired?

Answer: Automatic dependency injection.

134. What is @Qualifier?

Answer: Specifies exact bean.

135. What is @Primary?

Answer: Default bean selection.

136. What is AOP?

Answer: Aspect Oriented Programming.

137. Benefits of AOP?

Answer: Logging, Security, Transactions.

138. What is Aspect?

Answer: Cross-cutting concern.

139. What is Advice?

Answer: Action performed by Aspect.

140. What is Pointcut?

Answer: Expression selecting methods.

141. What is Join Point?

Answer: Execution point in application.

142. What is Spring JDBC?

Answer: Simplified JDBC support.

143. What is JdbcTemplate?

Answer: Executes JDBC operations.

144. What is Spring ORM?

Answer: Hibernate integration support.

145. Advantages of Spring?

Answer: Loose coupling, testability, modularity.

Spring Boot (146–165)

146. What is Spring Boot?

Answer: Simplified Spring framework.

147. Why Spring Boot?

Answer: Less configuration.

148. What is Starter Dependency?

Answer: Preconfigured dependency bundle.

149. What is @SpringBootApplication?

Answer: Main Spring Boot annotation.

150. Embedded Server?

Answer: Built-in Tomcat server.

151. application.properties?

Answer: Configuration file.

152. application.yml?

Answer: YAML configuration file.

153. What is Auto Configuration?

Answer: Automatic setup based on dependencies.

154. What is Spring Boot Actuator?

Answer: Monitoring and health endpoints.

155. What is DevTools?

Answer: Automatic restart support.

156. What is @Value?

Answer: Inject property values.

157. What is @Configuration?

Answer: Defines configuration class.

158. What is @Bean?

Answer: Creates Spring-managed bean.

159. What is CommandLineRunner?

Answer: Executes code after startup.

160. Spring Boot Profiles?

Answer: Environment-specific configurations.

161. What is Spring Data JPA?

Answer: Simplifies database access.

162. What is JpaRepository?

Answer: CRUD repository interface.

163. What is Paging and Sorting?

Answer: Data pagination and ordering.

164. What is Microservice?

Answer: Independently deployable service.

165. Monolith vs Microservice?

Answer: Single app vs multiple services.

REST API (166–180)

166. What is REST?

Answer: Representational State Transfer.

167. REST Principles?

Answer: Stateless, Client-Server, Cacheable.

168. What is API?

Answer: Application Programming Interface.

169. HTTP Methods?

Answer: GET, POST, PUT, DELETE.

170. GET Method?

Answer: Retrieve data.

171. POST Method?

Answer: Create resource.

172. PUT Method?

Answer: Update resource.

173. DELETE Method?

Answer: Delete resource.

174. What is JSON?

Answer: Lightweight data format.

175. What is XML?

Answer: Markup-based data format.

176. What is @RestController?

Answer: Creates REST endpoints.

177. What is @RequestMapping?

Answer: Maps URLs.

178. What is @PathVariable?

Answer: Reads value from URL path.

179. What is @RequestParam?

Answer: Reads query parameters.

180. What is ResponseEntity?

Answer: Represents HTTP response.

Java 8 Lambda & Stream API (181–200)

181. What is Java 8?

Answer: Major release introducing Lambda and Streams.

182. What is Lambda Expression?

Answer: Anonymous function.

`x -> x * 2`

183. Advantages of Lambda?

Answer: Less code and readability.

184. Functional Interface?

Answer: Interface with one abstract method.

185. Examples?

Answer: Runnable, Comparator.

186. What is @FunctionalInterface?

Answer: Marks functional interface.

187. What is Stream API?

Answer: Process collections efficiently.

188. Stream vs Collection?

Answer: Stream processes data; Collection stores data.

189. What is filter()?

Answer: Filters elements.

190. What is map()?

Answer: Transforms elements.

191. What is sorted()?

Answer: Sorts stream elements.

192. What is distinct()?

Answer: Removes duplicates.

193. What is collect()?

Answer: Converts stream result.

194. What is reduce()?

Answer: Combines elements into one result.

195. What is forEach()?

Answer: Iterates through stream.

196. What is Optional?

Answer: Prevents NullPointerException.

197. What is Method Reference?

Answer: Short form of lambda.

`System.out::println`

198. Types of Method References?

Answer: Static, Instance, Constructor.

199. Parallel Stream?

Answer: Multi-threaded stream processing.

200. Advantages of Stream API?

Answer: Concise code, better readability, easier parallel processing.

1. Difference between HashMap and ConcurrentHashMap?

HashMap	ConcurrentHashMap
Not Thread Safe	Thread Safe
Faster in single thread	Better in multi-thread
May throw ConcurrentModificationException	Safe during concurrent access

Interview Answer:

HashMap is not synchronized and is suitable for single-threaded applications. ConcurrentHashMap is thread-safe and allows multiple threads to read/write concurrently with better performance than Hashtable.

2. Why String is Immutable?

Reasons:

- Security
- Thread Safety
- String Pool Optimization
- HashMap Key Reliability

Interview Answer:

String is immutable because once created its value cannot be changed. This improves security, caching, thread safety, and ensures reliable behavior when used as HashMap keys.

3. Internal Working of HashMap?

Steps:

1. Key hashCode() calculated
2. Bucket index determined
3. Entry stored in bucket
4. If collision occurs → Linked List / Tree

Interview Answer:

HashMap uses hashing. It calculates hashCode() of the key, identifies a bucket, and stores the key-value pair. If multiple keys map to the same bucket, collision handling is done using LinkedList or Red-Black Tree.

4. Comparable vs Comparator?

Comparable

class Student implements Comparable<Student>

Comparator

class AgeComparator implements Comparator<Student>

Comparable	Comparator
-------------------	-------------------

Natural Sorting	Custom Sorting
compareTo()	compare()
Inside Class	Separate Class

Interview Answer:

Comparable is used for natural sorting while Comparator is used for custom sorting logic.

5. Checked vs Unchecked Exception?

Checked Exception

IOException
SQLException

Unchecked Exception

NullPointerException
ArithmeticException

Checked	Unchecke d
----------------	-----------------------

Compile Time	Runtime
-----------------	---------

Must Handle	Optional
-------------	----------

Interview Answer:

Checked exceptions are verified by compiler and must be handled. Unchecked exceptions occur at runtime and handling is optional.

6. throw vs throws?

throw

throw new Exception();

throws

public void test() throws IOException

throw

Used to throw
exception

throws

Used to declare
exception

Interview Answer:

throw is used to explicitly create and throw an exception, whereas throws declares possible exceptions in method signature.

7. Runnable vs Callable?**Runnable**

No Return Value

run()

No Checked
Exception

Callable

Returns Value

call()

Can Throw
Exception

Interview Answer:

Runnable is used when no result is required. Callable is used when task returns a result or throws checked exceptions.

8. Synchronization vs Concurrent Collections?**Synchronization**

synchronized method

Concurrent Collections

ConcurrentHashMap
CopyOnWriteArrayList

Synchronization	Concurrent Collections
------------------------	-------------------------------

Locks whole
object

Fine-grained locking

Slower

Faster

Interview Answer:

Synchronization locks shared resources whereas concurrent collections provide thread safety with better performance and scalability.

9. Statement vs PreparedStatement?

Statement

Statement st

PreparedStatement

PreparedStatement ps

Statement	PreparedStatement
Slow	Faster
SQL Injection Risk	Safe
Query Compiled Every Time	Precompiled

Interview Answer:

PreparedStatement improves performance and prevents SQL Injection attacks because queries are precompiled and parameters are handled safely.

10. ACID Properties?

A - Atomicity

All or Nothing

C - Consistency

Valid State

I - Isolation

Transactions Independent

D - Durability

Permanent Storage

Interview Answer:

ACID ensures reliable database transactions through Atomicity, Consistency, Isolation, and Durability.

11. Session vs Cookie?

Session	Cookie
---------	--------

Server Side	Client Side
-------------	-------------

More Secure	Less Secure
-------------	-------------

Larger Data	Small Data
-------------	------------

Interview Answer:

Session stores user information on the server, while cookies store data in the client browser.

12. Hibernate get() vs load()?

get()	load()
Immediate Fetch	Lazy Fetch
Returns Null	Throws Exception
Database Hit Immediately	Hit Later

Interview Answer:

get() retrieves object immediately. load() creates proxy object and loads data only when required.

13. Lazy vs Eager Loading?

Lazy Loading

Load only when needed.

Eager Loading

Load immediately.

Interview Answer:

Lazy loading improves performance by fetching data only when accessed, whereas eager loading retrieves related data immediately.

14. IoC vs Dependency Injection?

IoC

Control transferred to Spring Container.

DI

Way of implementing IoC.

Interview Answer:

IoC is a design principle where object creation is handled by container. Dependency Injection is the mechanism used to achieve IoC.

15. @Component vs @Service vs @Repository?

Annotation	Purpose
------------	---------

@Component	Generic Bean
------------	--------------

@Service	Business Layer
----------	----------------

@Repository	DAO Layer
-------------	-----------

Interview Answer:

All are Spring beans. @Service is for business logic, @Repository is for database operations, and @Component is generic.

16. Spring vs Spring Boot?

Spring	Spring Boot
--------	-------------

More Configuration	Auto Configuration
--------------------	--------------------

External Server	Embedded Server
-----------------	-----------------

Complex Setup	Easy Setup
---------------	------------

Interview Answer:

Spring Boot simplifies Spring development by providing auto-configuration, starter dependencies, and embedded servers.

17. JPA vs Hibernate?

JPA	Hibernate
-----	-----------

Specification	Implementation
---------------	----------------

Standard	ORM
API	Framework

Interview Answer:

JPA is a specification defining ORM standards. Hibernate is one of the most popular implementations of JPA.

18. PUT vs POST?

PUT	POST
-----	------

Update	Create
--------	--------

Idempotent	Not Idempotent
------------	-------------------

Interview Answer:

POST creates new resources, while PUT updates existing resources and can be safely repeated.

19. Lambda vs Anonymous Class?

Anonymous Class

```
Runnable r = new Runnable(){  
    public void run(){}  
};
```

Lambda

```
Runnable r = () -> {};
```

Lambda	Anonymous
--------	-----------

Less Code	More Code
-----------	-----------

Better Readability	Verbose
-----------------------	---------

Interview Answer:

Lambda expressions provide a concise way to implement functional interfaces compared to anonymous classes.

20. Stream API Advantages?**Advantages**

- ✓ Less Code
- ✓ Functional Programming
- ✓ Easy Filtering
- ✓ Parallel Processing
- ✓ Better Readability

Example:

```
List<Integer> list =  
Arrays.asList(1,2,3,4,5);
```

```
list.stream()  
    .filter(x -> x%2==0)  
    .forEach(System.out::println);
```

Interview Answer:

Stream API enables efficient data processing using functional operations like filter, map, and reduce, resulting in cleaner and more maintainable code.